E03 — Group Anagrams (Hash Table)

Experiment: 3 | **LeetCode:** #49 | **CO:** CO2, CO3 | **BT Level:** BT5

Problem Statement

Given an array of strings `strs`, group the anagrams together. You can return the answer in any order.

An **anagram** is a word formed by rearranging the letters of another word, using all original letters exactly once.

Example 1:

```
Input:  strs = ["eat","tea","tan","ate","nat","bat"]
Output: [["bat"],["nat","tan"],["ate","eat","tea"]]
```

Example 2:

```
Input:  strs = [""]
Output: [[""]]
```

Example 3:

```
Input:  strs = ["a"]
Output: [["a"]]
```

Concept Review: Anagram Detection

Two strings are anagrams if and only if they have **the same character frequencies**.

Key insight: Sorting two anagrams always produces the same string:

- "eat" sorted → "aet"
- "tea" sorted → "aet"
- "ate" sorted → "aet"

Use the **sorted string as a hash key** to group anagrams.

Solution 1: Sorted Key (Primary)

```
from collections import defaultdict

def groupAnagrams(strs):
    """
    Group anagrams using sorted string as hash key.
    Time: O(n · k log k)   n = number of strings, k = max string length
    Space: O(n · k)        for the hash table
```

```
    """
    anagram_map = defaultdict(list)

    for s in strs:
        key = tuple(sorted(s))   # Sorting takes O(k log k)
        anagram_map[key].append(s)

    return list(anagram_map.values())
```

Solution 2: Character Count Key (Optimal)

Instead of sorting (O(k log k)), build a **frequency tuple** in O(k):

```
def groupAnagrams_v2(strs):
    """
    Group anagrams using character frequency tuple as key.
    Time: O(n · k)   n = number of strings, k = max string length
    Space: O(n · k)
    """
    anagram_map = defaultdict(list)

    for s in strs:
        # Build a 26-element count tuple (one per lowercase letter)
        count = [0] * 26
        for c in s:
            count[ord(c) - ord('a')] += 1
        key = tuple(count)   # Hashable key: (2, 0, 0, ..., 1, ...)
        anagram_map[key].append(s)

    return list(anagram_map.values())
```

Detailed Trace

Input: `["eat", "tea", "tan", "ate", "nat", "bat"]`

Trace (Solution 1: sorted key)

String	sorted(s)	key (tuple)	anagram_map state
"eat"	"aet"	('a','e','t')	{('a','e','t'): ["eat"]}
"tea"	"aet"	('a','e','t')	{('a','e','t'): ["eat","tea"]}
"tan"	"ant"	('a','n','t')	{('a','e','t'): [...], ('a','n','t'): ["tan"]}
"ate"	"aet"	('a','e','t')	{('a','e','t'): ["eat","tea","ate"], ...}
"nat"	"ant"	('a','n','t')	{('a','n','t'): ["tan","nat"], ...}
"bat"	"abt"	('a','b','t')	{('a','b','t'): ["bat"], ...}

Result: `[["eat","tea","ate"], ["tan","nat"], ["bat"]]` ✓

Trace (Solution 2: frequency key)

String	Frequency count (a-z positions)	Key
"eat"	a=1, e=1, t=1	(1,0,0,0,1,0,...,1,0,0)
"tea"	a=1, e=1, t=1	(1,0,0,0,1,0,...,1,0,0) ← same key
"bat"	a=1, b=1, t=1	(1,1,0,0,0,...,1,0,0)

Complexity Comparison

Solution	Time	Space	Notes
Sort key	O(n · k log k)	O(n · k)	Simple, readable
Count key	O(n · k)	O(n · k)	Faster for large k

Python Counter Approach (for understanding)

```
from collections import Counter

def groupAnagrams_counter(strs):
    """Using Counter for readability (not hashable directly — need tupl
    anagram_map = defaultdict(list)
    for s in strs:
        key = tuple(sorted(Counter(s).items()))  # e.g., (('a',1),('e',
        anagram_map[key].append(s)
    return list(anagram_map.values())
```

Test Suite

```
def test_group_anagrams():
    test_cases = [
        (["eat","tea","tan","ate","nat","bat"],
         [["eat","tea","ate"], ["tan","nat"], ["bat"]]),
        ([""], [[""]]),
        (["a"], [["a"]]),
        (["ab","ba","abc","cab","bca"], [["ab","ba"],["abc","cab","bca"
    ]

    for strs, expected in test_cases:
        result = groupAnagrams(strs[:])
        # Compare as sorted sets (order of groups doesn't matter)
        result_sorted = sorted([sorted(g) for g in result])
        expected_sorted = sorted([sorted(g) for g in expected])
        status = "PASS" if result_sorted == expected_sorted else "FAIL"
        print(f"{status}: {strs} → {result_sorted}")

test_group_anagrams()
```

Key Takeaways

- **Canonical form** (sorted string or frequency count) is the standard technique to detect anagrams.
- Using a **hash table** (dict) with the canonical form as key achieves O(1) grouping per string.
- `defaultdict(list)` simplifies the grouping pattern — no need to check key existence.
- For large alphabets or Unicode, the character count approach scales better than sorting.

References

- LeetCode #49 — Group Anagrams
- Skiena, *The Algorithm Design Manual*, Ch. 4.2 (Dictionaries and Hashing)